CORPORACIÓN UNIVERSITARIA IBEROAMERICANA

INGENIERÍA DE SOFTWARE

ACTIVIDAD 6 – EJERCICIOS PRÁCTICOS DE APLICACIÓN DE TÉCNICAS Y HERRAMIENTAS DE CALIDAD

RESULTADOS DEL ANÁLISIS ESTÁTICO DE CÓDIGO DE UNA APLICACIÓN MÓVIL DE GESTIÓN FINANCIERA

Presentado por:

Lizeth Valentina Benítez

Asignatura: Calidad de software

Docente: William Ruiz

Curso: 24082026_C1_202634

1. Introducción

En el desarrollo de software no es suficiente con que un programa funcione una vez. También es necesario comprobar cómo responde ante diferentes situaciones, identificar posibles problemas y revisar si el código puede mejorarse. Esto cobra especial importancia en los sistemas financieros, porque una operación mal calculada o una transferencia procesada de forma incorrecta puede afectar directamente al usuario.

En la Actividad 4 se trabajó un caso relacionado con una aplicación móvil de gestión financiera que presentaba dificultades de rendimiento, errores en las transacciones y problemas de usabilidad. Para continuar con ese tema, en esta actividad preparé un prototipo pequeño en Python que permite calcular saldos y simular transferencias. El propósito fue llevar algunos de los conceptos de calidad del software a un ejercicio práctico y comprobar qué resultados podían obtenerse con herramientas de evaluación.

Primero preparé pruebas automatizadas para revisar el funcionamiento de las operaciones implementadas. Después generé un informe de cobertura y consulté los resultados del análisis en SonarQube. Este proceso me permitió observar que cada herramienta aporta información diferente: las pruebas comprueban situaciones específicas, mientras que las métricas ayudan a revisar algunos aspectos del código.

En las pruebas realizadas, los cuatro casos definidos terminaron correctamente. Por su parte, el panel de SonarQube mostró un Quality Gate aprobado, una cobertura del 76,2 %, una duplicación del 0,0 % y cero incidencias abiertas en el análisis consultado. Aunque estos resultados son favorables para el prototipo, no significan que el programa esté libre de todos los errores posibles.

El alcance de esta actividad es limitado. El programa desarrollado no es la aplicación móvil original ni cuenta con todas las funciones de un sistema financiero real. Por esa razón, el informe se concentra en los resultados que pude comprobar y en las mejoras que considero necesarias para continuar evaluando el código.

2. Objetivos

2.1. Objetivo general

Evaluar la calidad de un prototipo académico de gestión financiera mediante pruebas automatizadas, medición de cobertura y análisis estático con SonarQube, para identificar sus resultados actuales y proponer mejoras a partir de la evidencia obtenida.

2.2. Objetivos específicos

- Comprobar el funcionamiento de las operaciones financieras implementadas mediante pruebas automatizadas.
- Obtener y revisar la cobertura del código durante la ejecución de las pruebas.
- Consultar los indicadores de calidad disponibles en SonarQube.
- Interpretar los resultados teniendo en cuenta el tamaño y las limitaciones del prototipo.
- Proponer acciones que permitan ampliar las pruebas y fortalecer la evaluación del software.
- Organizar las evidencias técnicas para que los resultados puedan ser revisados y reproducidos.

3. Herramientas utilizadas y alcance del proyecto

3.1. Herramientas utilizadas

Para realizar el ejercicio utilicé herramientas que me permitieron ejecutar el programa, comprobar sus resultados y consultar métricas de calidad.

Herramienta	Uso en la actividad
Python 3.13.1	Lenguaje utilizado para desarrollar y ejecutar el prototipo.
pytest 9.1.1	Ejecución de las pruebas automatizadas del proyecto.
coverage.py	Generación del informe de cobertura del código.
SonarQube Community Build	Consulta de indicadores y resultados del análisis estático.
PowerShell	Ejecución de comandos y revisión de los archivos del proyecto.

Las funciones de pytest y coverage.py se relacionan con la ejecución de pruebas y la medición de cobertura, respectivamente, de acuerdo con su documentación oficial (Coverage.py, s. f.; pytest, s. f.). SonarQube se utilizó para revisar los indicadores publicados por la instancia de análisis (SonarSource, s. f.).

La versión de Python se verificó durante el trabajo. La versión exacta de SonarQube no quedó registrada en las evidencias disponibles, por lo que no se incluye una versión específica de esa herramienta.

3.2. Descripción del prototipo

El proyecto se encuentra en la carpeta actividad6_calidad_software. El archivo principal es app_financiera.py, donde desarrollé tres funciones para representar operaciones básicas.
La función calcular_saldo recibe un saldo inicial, una lista de ingresos y una lista de gastos. Con esos valores obtiene el saldo final. La función procesar_transferencia comprueba si el monto es mayor que cero y si el saldo alcanza para realizar la operación. Por último, mostrar_resumen presenta el saldo disponible y el mensaje correspondiente.
También preparé el archivo test_app_financiera.py, donde están definidos los casos que permiten revisar las dos primeras funciones. De esta manera, pude comprobar el comportamiento del programa sin tener que ejecutar manualmente cada operación.

3.3. Alcance y limitaciones

El ejercicio se concentra en las operaciones implementadas, las pruebas automatizadas, la cobertura y los resultados del análisis de SonarQube. No incluye una aplicación móvil completa, una base de datos, autenticación de usuarios, conexión con entidades bancarias ni procesamiento de transacciones reales.
Esta diferencia es importante porque el caso original corresponde a una aplicación financiera con dificultades que requieren una evaluación más amplia. El prototipo permite practicar algunas técnicas de calidad, pero no reproduce todas las condiciones de ese sistema.
Por tanto, los resultados que presento corresponden exclusivamente al código analizado. Para evaluar una aplicación real sería necesario disponer de su código fuente, conocer sus requisitos y realizar pruebas funcionales, de seguridad, rendimiento y usabilidad, entre otras

4. Metodología

4.1. Preparación del entorno

Organicé los archivos del proyecto en una carpeta local y verifiqué que Python pudiera ejecutarse desde PowerShell. Durante la preparación encontré una dificultad con la ruta del entorno, porque el directorio Scripts no contenía el ejecutable python.exe que había intentado utilizar
Para continuar utilicé el comando py, que permitió ejecutar Python y las herramientas instaladas. Esta situación me llevó a comprobar los comandos antes de continuar con las pruebas, ya que un problema de configuración puede impedir que el análisis se realice correctamente o puede generar resultados que no correspondan al proyecto esperado.

4.2. Preparación de las pruebas

Definí cuatro casos para comprobar los resultados de las operaciones financieras. Los escenarios se eligieron porque permiten revisar tanto una operación normal como algunas condiciones que deberían impedir una transferencia.

Código	Prueba realizada	Resultado esperado
PR-01	Calcular el saldo después de sumar ingresos y descontar gastos.	Obtener un saldo de 125.000.
PR-02	Procesar una transferencia cuando existe saldo suficiente.	Descontar 20.000 y mostrar el mensaje de operación procesada.
PR-03	Intentar una transferencia superior al saldo disponible.	Conservar el saldo y mostrar el mensaje de saldo insuficiente.
PR-04	Intentar una transferencia con monto igual a cero.	Conservar el saldo y mostrar el mensaje de validación.

Estos casos permiten comprobar el comportamiento básico de las funciones, pero no cubren todas las entradas posibles. Por ejemplo, todavía sería necesario revisar qué sucede con montos negativos, valores que no sean numéricos o cantidades extremas

4.3. Ejecución de las pruebas automatizadas

Para ejecutar los casos utilicé el siguiente comando en PowerShell:

```
py -m pytest -v test_app_financiera.py
```

En una primera ejecución general, pytest también encontró pruebas pertenecientes a paquetes instalados en el directorio Lib. Por eso, el resultado inicial incluía casos que no formaban parte de mi prototipo.

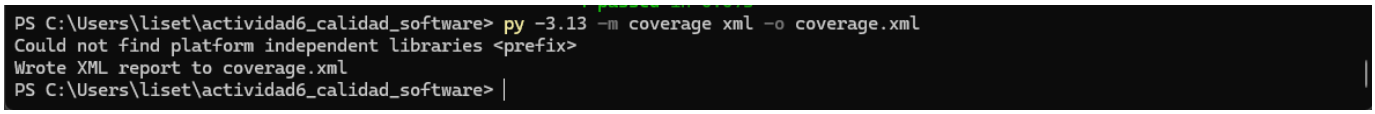
Para obtener un resultado específico, ejecuté únicamente el archivo test_app_financiera.py. Esta segunda ejecución recopiló cuatro pruebas y las cuatro terminaron correctamente. Por esa razón, utilicé este resultado para el informe y no el resumen de la ejecución general.

4.4. Generación del informe de cobertura

Después de ejecutar las pruebas, generé un informe de cobertura en formato XML mediante el comando:

py -3.13 -m coverage xml -o coverage.xml

Figura 2
Generación del informe de cobertura en formato XML



Nota. La terminal confirma la generación del archivo coverage.xml, utilizado para proporcionar información de cobertura al análisis del proyecto. Elaboración propia (2026).

La terminal confirmó que se había escrito el archivo coverage.xml. Este informe se utilizó para proporcionar información de cobertura al análisis del proyecto.
Posteriormente, SonarQube mostró una cobertura del 76,2 %. El indicador permite identificar que no todo el código contabilizado quedó cubierto por la ejecución de las pruebas. Para decidir cómo mejorarlo, es necesario revisar cuáles líneas o rutas no se ejecutaron y comprobar si corresponden a situaciones relevantes.

4.5. Consulta de los resultados en SonarQube

Una vez ejecutado el análisis, revisé el panel del proyecto *Analisis de Calidad - Actividad 6*. Allí consulté el estado del Quality Gate, las incidencias abiertas, la cobertura, la duplicación y los indicadores de seguridad, confiabilidad y mantenibilidad.
Para interpretar estos resultados tuve en cuenta que SonarQube informa sobre el código y las reglas incluidas en el análisis configurado. Por ello, un indicador sin incidencias no debe interpretarse como una garantía de que nunca se presentarán errores.

5. Criterios de evaluación

Para organizar la revisión establecí criterios relacionados con las pruebas, las métricas y el estado general del análisis.

Criterio	Qué revisé	Evidencia
Funcionamiento	Si las funciones entregaban los resultados esperados en los escenarios definidos.	Salida de pytest.
Cobertura	Qué proporción del código contabilizado fue ejecutada durante las pruebas.	Informe coverage.xml y panel de SonarQube.
Duplicación	Si la herramienta reportaba bloques de código duplicado.	Panel de SonarQube.
Seguridad	Si el análisis mostraba incidencias de seguridad abiertas.	Indicadores de SonarQube.
Confiabilidad	Si se reportaban incidencias clasificadas en esta dimensión.	Indicadores de SonarQube.
Mantenibilidad	Si se reportaban incidencias relacionadas con el mantenimiento del código.	Indicadores de SonarQube.
Quality Gate	Si se cumplían las condiciones configuradas para el análisis.	Estado mostrado por SonarQube.

Estos criterios no tienen el mismo alcance. Las pruebas automatizadas comprueban resultados concretos, mientras que el análisis estático revisa el código de acuerdo con las reglas activas. Ninguno de los dos sustituye por completo una revisión manual ni las pruebas especializadas que podrían requerirse en un sistema financiero.

6. Resultados del análisis

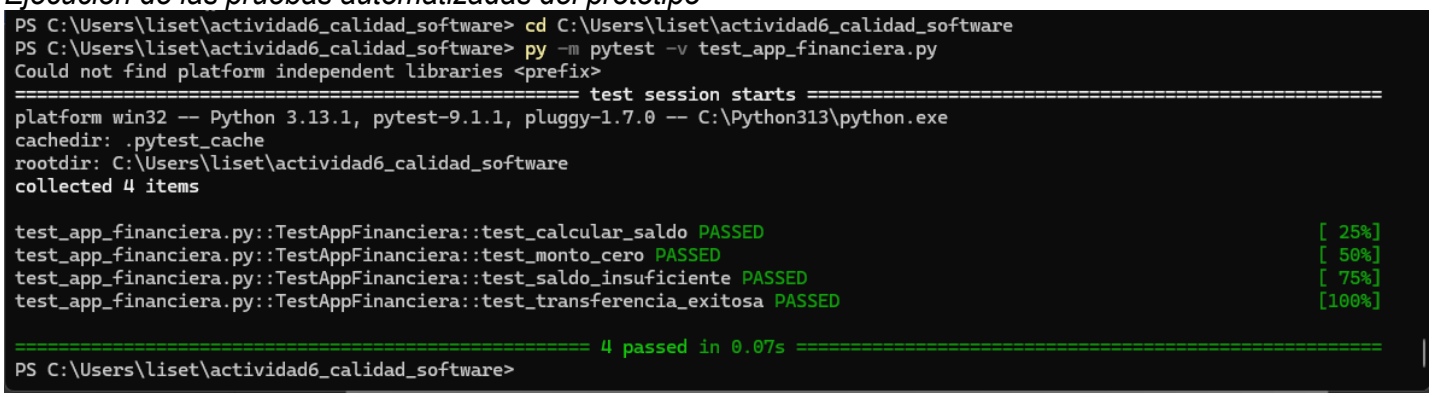
6.1. Resultados de las pruebas automatizadas

Al ejecutar únicamente test_app_financiera.py, pytest recopiló cuatro pruebas. La salida de la terminal mostró el estado PASSED en cada una y finalizó con el resumen de cuatro pruebas aprobadas.

Indicador	Resultado
Pruebas ejecutadas	4
Pruebas aprobadas	4
Pruebas fallidas	0
Porcentaje de aprobación	100 %

Figura 1

Ejecución de las pruebas automatizadas del prototipo



Nota. La ejecución de pytest muestra que los cuatro casos definidos en el archivo test_app_financiera.py fueron aprobados. Elaboración propia (2026).

El porcentaje se obtiene dividiendo las pruebas aprobadas entre el total ejecutado y multiplicando por cien. En este caso, las cuatro pruebas fueron aprobadas.
Este resultado me permitió comprobar que las operaciones respondieron como esperaba en los escenarios que preparé. Sin embargo, no sería correcto afirmar que el programa no contiene errores, porque solo se evaluaron cuatro situaciones. Para aumentar la confianza en el código, sería necesario agregar más casos y volver a ejecutar las pruebas

6.2. Resultados observados en SonarQube

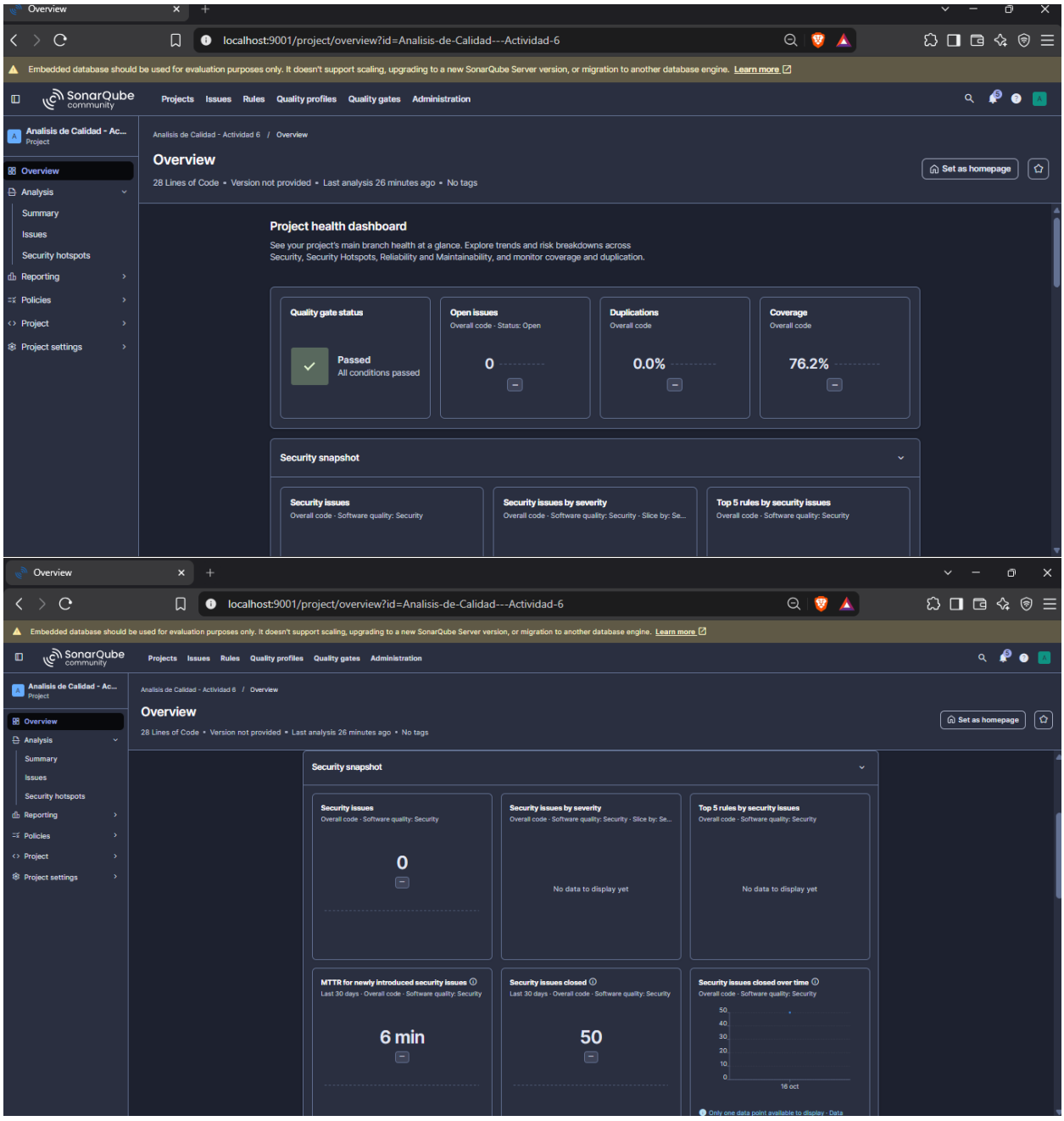
El panel del proyecto mostró los siguientes indicadores durante la consulta realizada

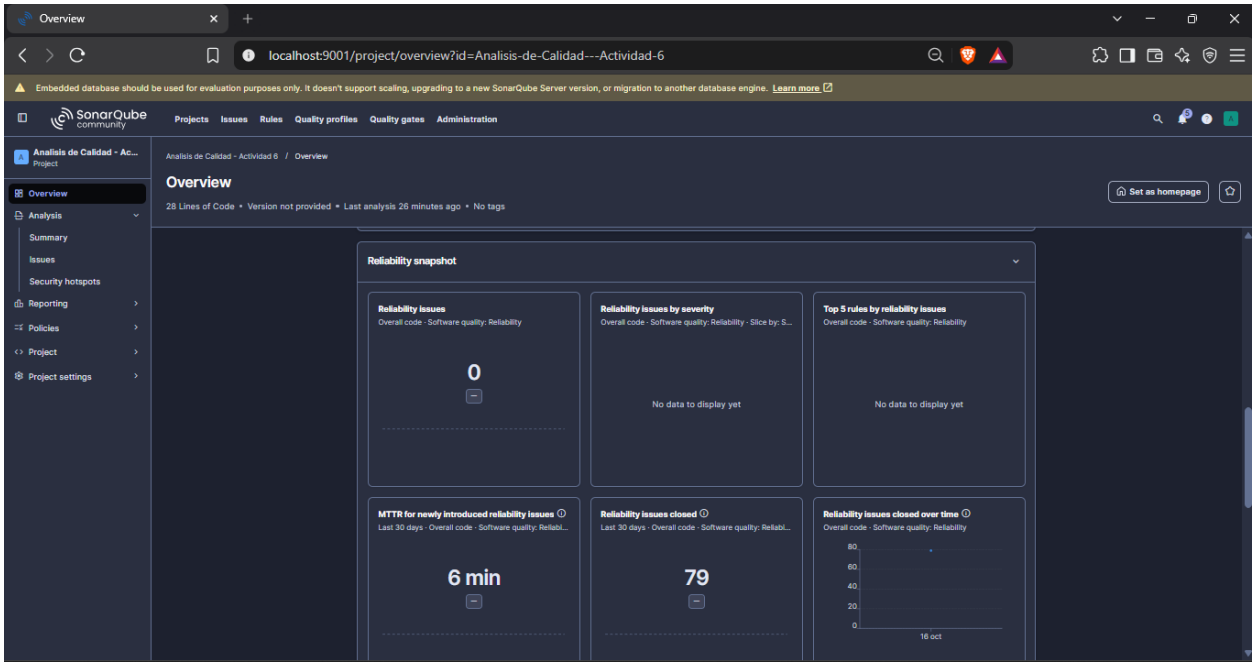
Indicador	Resultado observado
Estado del Quality Gate	Passed

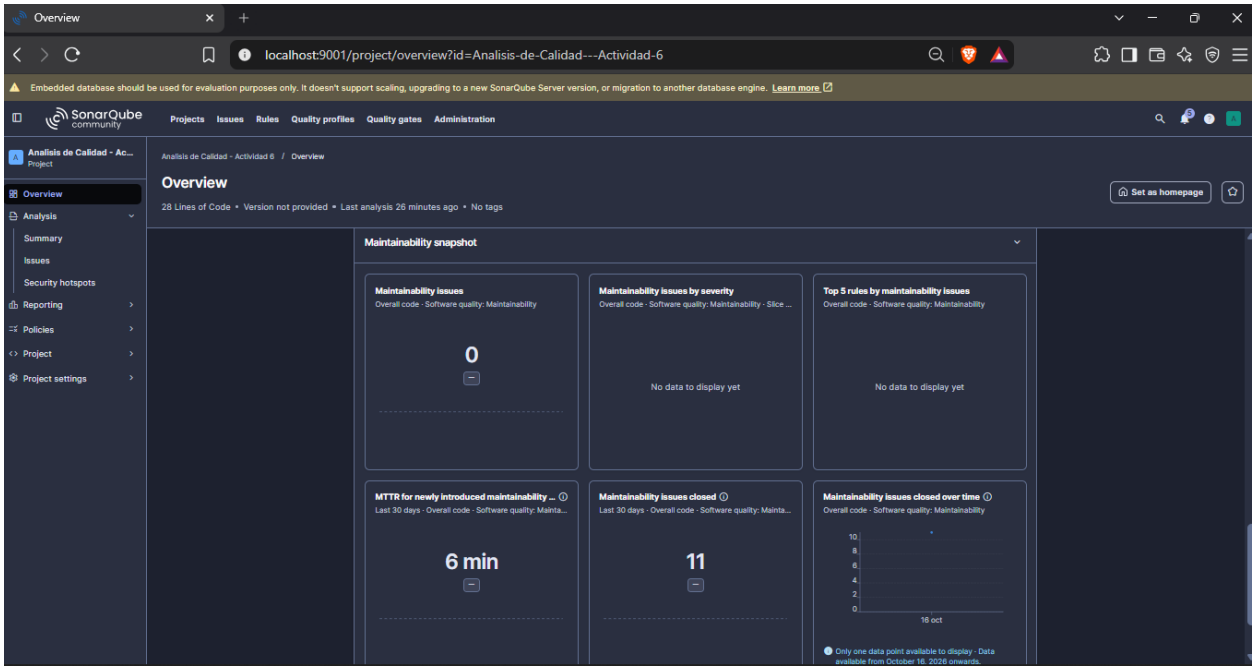
Incidencias abiertas	0
Cobertura	76,2 %
Duplicación	0,0 %
Incidencias de seguridad	0
Incidencias de confiabilidad	0
Incidencias de mantenibilidad	0
Líneas de código	28

Nota. Los valores corresponden al panel consultado de SonarQube para el proyecto analizado. Deben acompañarse de las capturas correspondientes para permitir su comprobación. La interpretación se limita al alcance y a la configuración de ese análisis.

Figura 3
Resultado general del análisis de calidad en SonarQube







Nota. La captura muestra el resultado del análisis del proyecto y el estado del Quality Gate en SonarQube. Elaboración propia (2026).

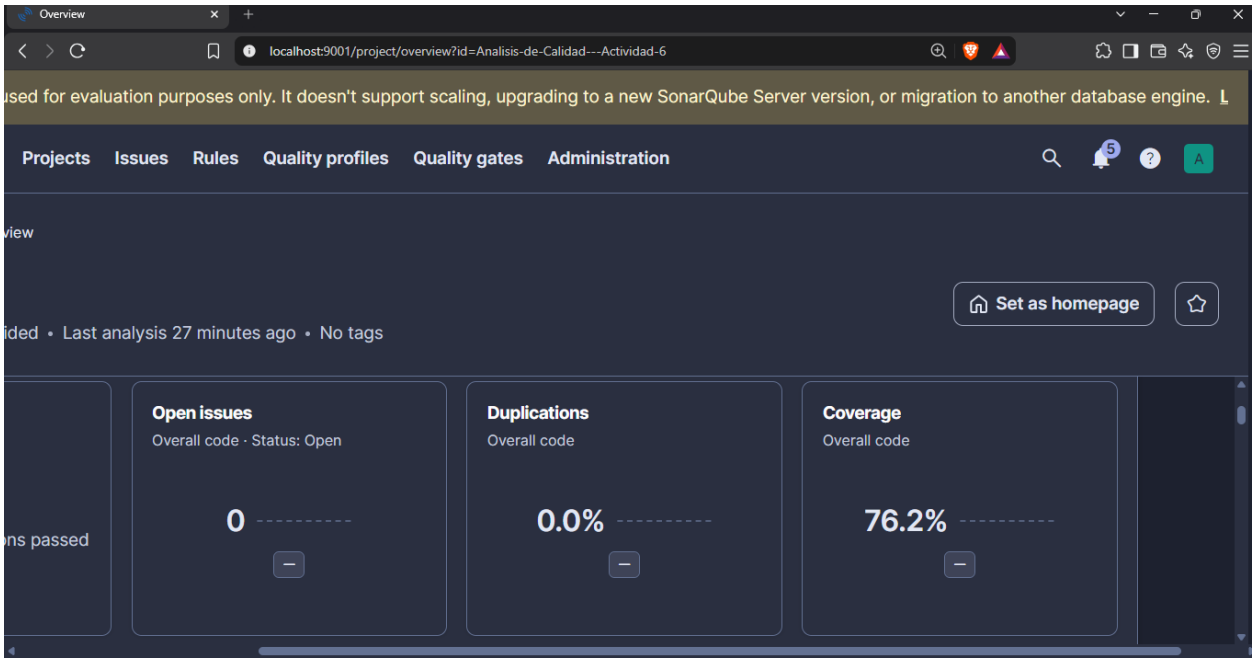
6.3. Análisis de la cobertura

La cobertura reportada fue del 76,2 %. Este fue uno de los indicadores que más me interesó revisar, porque muestra que todavía hay código que no se ejecutó durante las pruebas instrumentadas.

Para mejorar este resultado, considero necesario identificar las líneas o rutas que quedaron pendientes y preparar casos que permitan comprobar su comportamiento. No se trata solamente de elevar el porcentaje, sino de verificar que el programa responda correctamente ante situaciones adicionales.

También es importante tener presente que la cobertura no indica por sí sola si las pruebas están bien diseñadas. Una línea puede ejecutarse sin que la prueba compruebe adecuadamente el resultado. Por eso, el siguiente paso debería combinar la revisión de las líneas no cubiertas con la evaluación de los casos que todavía hacen falta.

Figura 4
Métricas de cobertura y duplicación del código



Nota. SonarQube reportó una cobertura del 76,2 % y una duplicación del 0,0 % en el análisis del proyecto. Elaboración propia (2026).

6.4. Análisis de la duplicación

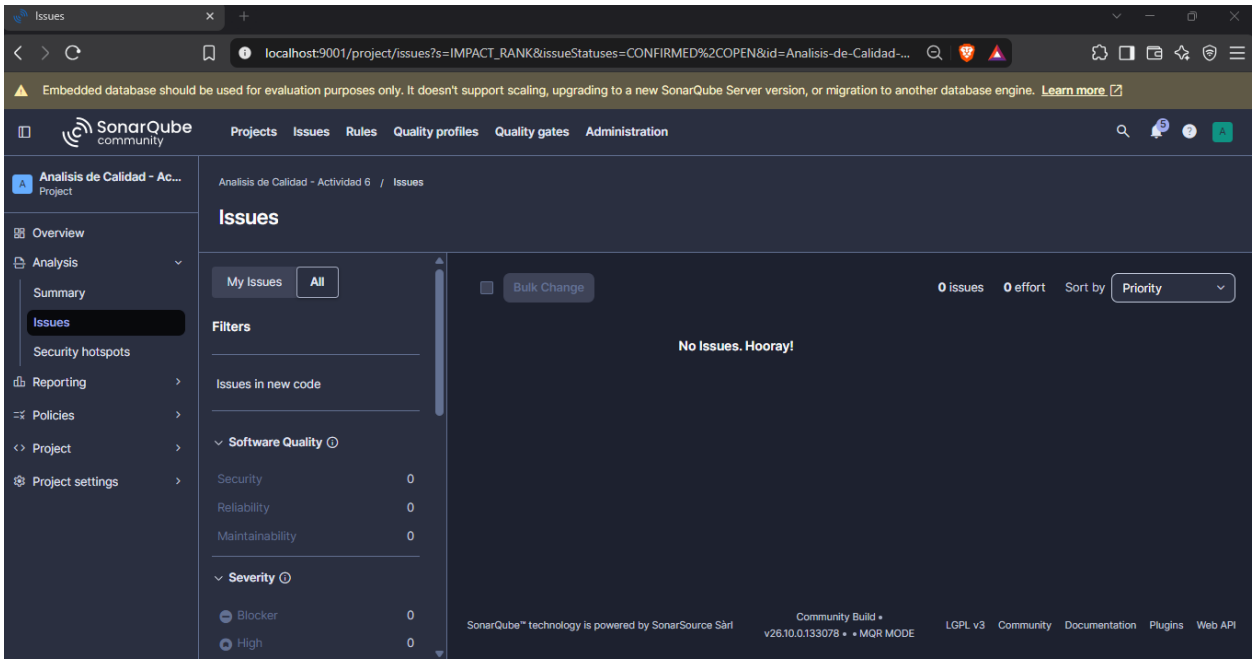
SonarQube mostró un indicador de duplicación del 0,0 %. En el análisis realizado, la herramienta no reportó código duplicado según la métrica presentada.
Este resultado es favorable, pero debe interpretarse teniendo en cuenta que el prototipo es pequeño y el panel registra 28 líneas de código. Un proyecto de este tamaño no permite sacar conclusiones sobre cómo se comportaría una aplicación completa con muchas más funciones. Si el sistema creciera, sería conveniente volver a revisar este indicador y comprobar si las nuevas funcionalidades introducen bloques repetidos que puedan reorganizarse.

6.5. Análisis de las incidencias

En el panel consultado no aparecían incidencias abiertas de seguridad, confiabilidad o mantenibilidad. Esto indica que el análisis no estaba mostrando hallazgos abiertos en esas categorías dentro del alcance evaluado.
Aun así, es necesario evitar una interpretación demasiado amplia. La ausencia de incidencias en SonarQube no confirma que el programa sea completamente seguro ni que no existan errores funcionales. La herramienta examina el código mediante las reglas y condiciones configuradas, mientras que otros problemas podrían requerir pruebas diferentes o una revisión manual.
En particular, el prototipo no contiene autenticación real ni procesa datos bancarios. Por ello, no permite evaluar todos los riesgos que tendría una aplicación financiera utilizada por personas

reales.

Figura 5
Resumen de incidencias detectadas en SonarQube



Nota. En el análisis consultado, SonarQube reportó cero incidencias abiertas y cero problemas en las categorías de seguridad, fiabilidad y mantenibilidad. Estos resultados corresponden al alcance del prototipo evaluado. Elaboración propia (2026).

6.6. Análisis del Quality Gate

El Quality Gate apareció como Pased, es decir, aprobado. Este resultado indica que el proyecto cumplió las condiciones configuradas para esa evaluación.
Lo considero un resultado positivo, aunque no significa que la calidad sea perfecta. El estado debe analizarse junto con los demás indicadores y con el alcance reducido del proyecto. En este caso, la cobertura del 76,2 % muestra que todavía hay espacio para ampliar las pruebas, incluso cuando el qualit Gate aparece aprobado.

6.7. Balance general de los resultados

Después de revisar las pruebas y el panel de SonarQube, identifiqué tres resultados favorables: las cuatro pruebas automatizadas fueron aprobadas, la duplicación reportada fue del 0,0 % y el quality gate quedó aprobado. Además, el panel no mostró incidencias abiertas en las categorías consultadas

La principal oportunidad de mejora identificada es la cobertura del 76,2 %. Para avanzar, sería necesario conocer con precisión qué partes del código quedaron sin ejecutar y añadir pruebas que permitan revisarlas.

En conjunto, los resultados son favorables para el prototipo académico. Sin embargo, el tamaño del código y el número limitado de pruebas impiden presentar este diagnóstico como una evaluación integral de una aplicación financiera real

7. Análisis de impacto

7.1. Funcionamiento de las operaciones

Las cuatro pruebas permiten comprobar que las funciones respondieron de acuerdo con los resultados esperados para los casos definidos. Esto aporta evidencia sobre el cálculo de saldo, la transferencia exitosa, el rechazo por saldo insuficiente y la validación de un monto igual a cero.
Sin embargo, todavía hay situaciones que no se han comprobado. Por ejemplo, sería útil verificar qué sucede cuando el monto de la transferencia es negativo o cuando el saldo disponible es exactamente igual al monto solicitado. Estos casos permitirían revisar con más detalle las condiciones de validación.

7.2. Seguridad

Aunque el panel no mostró incidencias de seguridad abiertas, el prototipo no implementa mecanismos reales de autenticación, autorización ni protección de datos financieros. Por esta razón, no puedo concluir que una aplicación bancaria completa estaría protegida con base en estos resultados.
Si el proyecto incorporara usuarios, información sensible o conexión con servicios externos, sería necesario ampliar la evaluación con pruebas y revisiones específicas de seguridad.

7.3. Mantenibilidad

La ausencia de duplicación reportada es un resultado favorable para el código analizado. También es positivo que el panel no muestre incidencias abiertas de mantenibilidad.
No obstante, el proyecto tiene un tamaño reducido. A medida que se agreguen funcionalidades, será necesario comprobar si el código sigue siendo fácil de entender, modificar y probar. Mantener las funciones claras y las pruebas actualizadas ayudaría a detectar problemas durante las siguientes etapas de desarrollo.

7.4. Rendimiento y usabilidad

En esta actividad no se registraron tiempos de respuesta ni se realizaron pruebas de carga. Tampoco se evaluó la experiencia de personas utilizando una interfaz móvil. Por eso, los resultados obtenidos no permiten afirmar que se hayan solucionado las dificultades de rendimiento o usabilidad mencionadas en el caso original.
Para revisar el rendimiento sería necesario definir operaciones representativas, medir sus tiempos de ejecución y repetir las mediciones en condiciones documentadas. La usabilidad, por su parte, requeriría una interfaz y tareas concretas que pudieran probarse con usuarios.

8. Recomendaciones de mejora

8.1. Agregar más pruebas

Recomiendo ampliar los escenarios actuales con casos límite, como una transferencia cuyo monto sea igual al saldo disponible, un monto negativo y entradas con tipos de datos inesperados. Cada caso debe tener un resultado esperado claramente definido.

8.2. Revisar el código no cubierto

La cobertura del 76,2 % indica que es conveniente revisar el informe generado y localizar las partes que no se ejecutaron durante las pruebas. A partir de esa revisión se podrán crear

nuevos casos para comprobar las rutas que sean relevantes

8.3. Ejecutar SonarQube después de los cambios

Cada vez que se realicen modificaciones importantes, recomiendo repetir el análisis y conservar los resultados. Así será posible comprobar si los cambios afectan la cobertura, la duplicación o las incidencias reportadas.

8.4. Ampliar la evaluación de seguridad

Si el prototipo evoluciona hacia una aplicación con usuarios reales, se deberán revisar la autenticación, los permisos, el almacenamiento de información y la comunicación con otros servicios. Los resultados actuales no son suficientes para evaluar esos aspectos.

8.5. Medir el rendimiento

Para estudiar los problemas de lentitud del caso original, se recomienda definir escenarios de prueba y registrar tiempos de respuesta. Los resultados deberán documentarse antes y después de aplicar mejoras para poder compararlos.

8.6. Conservar las evidencias

Las capturas de la terminal, el informe de cobertura y el panel de SonarQube deben guardarse en una carpeta organizada. Estas evidencias permiten respaldar los datos del informe y facilitan la revisión del trabajo.

9. Plan de mejora

A partir de los resultados obtenidos, propongo el siguiente plan para continuar evaluando el prototipo.

Acción propuesta	Prioridad	Forma de comprobar el avance
Crear pruebas para casos límite.	Alta	Revisar el nuevo informe de pytest.
Identificar las líneas no cubiertas.	Alta	Consultar el informe de cobertura.
Repetir el análisis estático después de los cambios.	Alta	Comparar los indicadores de SonarQube.
Revisar posibles riesgos de seguridad cuando se agreguen nuevas funciones.	Alta	Documentar las pruebas y los hallazgos.
Medir el rendimiento de las operaciones.	Media	Registrar y comparar tiempos de respuesta.
Mantener las evidencias organizadas.	Media	Verificar que cada resultado tenga su captura o archivo de soporte.

Este plan plantea acciones posteriores; no significa que todas se hayan realizado durante la actividad actúa

10. Discusión

La práctica me permitió comprobar que las pruebas automatizadas y el análisis estático sirven para revisar aspectos distintos del mismo proyecto. Las pruebas de pytest me mostraron si las funciones respondían como esperaba en los casos preparados, mientras que SonarQube presentó indicadores que ayudaron a revisar el estado del código.
Las cuatro pruebas aprobadas son un resultado favorable, pero también me permitieron reconocer una limitación: probar unas pocas situaciones no es suficiente para cubrir todas las formas en que una operación financiera podría ejecutarse. Por eso, considero necesario ampliar los escenarios antes de confiar en el comportamiento del programa ante entradas diferentes.

La cobertura del 76,2 % también deja una tarea pendiente. Aunque el Quality Gate esté aprobado, todavía es importante revisar qué partes del código no se ejecutaron durante las pruebas. Esto demuestra que una sola métrica no debería utilizarse como criterio definitivo para decidir si un programa tiene buena calidad.
Otro aspecto que tuve en cuenta fue el tamaño del prototipo. El panel muestra 28 líneas de código, de modo que el resultado de duplicación del 0,0 % debe interpretarse dentro de ese contexto. No sería adecuado utilizarlo para afirmar que una aplicación financiera más compleja tendría el mismo comportamiento.
Finalmente, este ejercicio permitió continuar el caso de estudio de la Actividad 4 desde una perspectiva práctica, pero no comprobó los problemas de la aplicación móvil original. Para hacerlo sería necesario analizar el código real y realizar las pruebas correspondientes. La diferencia entre lo que se observó y lo que todavía debe evaluarse es importante para que el informe presente conclusiones claras y respaldadas por evidencias.

11. Conclusiones

Durante esta actividad preparé un prototipo en Python y utilicé pruebas automatizadas y SonarQube para revisar algunos aspectos de su calidad. Las cuatro pruebas implementadas fueron aprobadas, lo que permitió comprobar el resultado de los escenarios de cálculo de saldo y procesamiento de transferencias.

El panel de SonarQube mostró un quality Gate aprobado, una cobertura del 76,2 %, una duplicación del 0,0 % y cero incidencias abiertas en las categorías consultadas. Estos datos son favorables dentro del alcance del ejercicio, aunque no permiten afirmar que el programa esté libre de errores ni que cumpla todos los requisitos de una aplicación financiera real.
La cobertura es el indicador que considero prioritario revisar en una siguiente etapa. Identificar las partes del código que no se ejecutaron y preparar nuevas pruebas ayudaría a comprobar más situaciones. También sería conveniente incluir casos límite y repetir el análisis después de cada cambio importante.

La actividad me permitió entender que las herramientas de calidad no reemplazan el análisis del desarrollador. Sus resultados necesitan interpretarse de acuerdo con el tamaño del proyecto, las pruebas realizadas y las reglas configuradas. Por esa razón, considero que el prototipo presenta resultados iniciales favorables, pero todavía requiere pruebas adicionales para lograr una evaluación más completa.

En conclusión, el trabajo demuestra la aplicación práctica de técnicas de verificación y análisis estático sobre un programa académico. Sus resultados deben entenderse como un diagnóstico limitado del prototipo, no como una certificación de una aplicación bancaria ni como evidencia de que se hayan resuelto los problemas del caso original.

12. Referencias Bibliograficas

- Coverage.py. (s. f.). *Coverage.py documentation*. https://coverage.readthedocs.io/
- Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería de software: Un enfoque práctico*. McGraw-Hill Interamericana.
- pytest. (s. f.). *pytest documentation*. https://docs.pytest.org/
- SonarSource. (s. f.). *SonarQube documentation*. https://docs.sonarsource.com/sonarqube/